

README

ИД34АКОС

Комкова Полина Дмитриевна БПИ244

Вариант 14

Условие задачи

В магазине работают **два отдела**, каждый отдел обладает уникальным ассортиментом. В **каждом** отделе работает **один продавец**.

В магазин ходят исключительно забывчивые **покупатели**, поэтому каждый покупатель носит с собой **список из K товаров**, которые желает купить. Покупатель приобретает товары точно в том порядке, в каком они записаны в его списке. Число товаров в списке **от одного до пяти**. При этом товары в списке расположены в случайном порядке, что заставляет покупателя многократно переходить от отдела к отделу, если это требуется для совершения покупок.

Продавец может обслужить **только одного** покупателя за раз. Покупатель, вставший в очередь, **засыпает** пока не дойдет до продавца. Продавец **засыпает**, если в его отделе нет покупателей, и просыпается, если появится хотя бы один.

Требуется создать многопоточное приложение, моделирующее работу магазина в течение рабочего дня. Каждый покупатель и продавец задаются отдельным **поток**

Сценарий задачи

Покупатель:

- Приходит в магазин с заранее составленным списком покупок (*от 1 до 5 товаров*)
- Товары в списке расположены в случайном порядке
- Покупает товары строго в том порядке, в котором они записаны в списке
- Для каждого товара из списка:
 - Определяет, в каком отделе (*1 или 2*) находится нужный товар
 - Подходит к соответствующему отделу и встает в очередь
 - Засыпает, ожидая своей очереди

- Просыпается, когда продавец его обслуживает
- Получает товар и переходит к следующему пункту списка
- После покупки всех товаров покидает магазин

Продавец:

- Работает в своем отделе (*отдел 1 или отдел 2*)
- Находится в ожидании, если в его отделе нет покупателей
- Просыпается, когда в отделе появляется хотя бы один покупатель
- Обслуживает покупателей строго по очереди (FIFO)
- Одновременно может обслуживать только одного покупателя
- Время обслуживания одного покупателя составляет *1-2 секунды*
- После обслуживания покупателя переходит к следующему в очереди или засыпает, если очередь пуста
- Завершает работу, когда рабочий день закончен и очередь пуста

Магазин:

- Содержит два отдела
- Каждый отдел имеет свою очередь покупателей
- Рабочий день продолжается до тех пор, пока все покупатели не завершат свои покупки
- Поддерживает корректное завершение работы при получении сигнала прерывания (SIGINT, SIGTERM)

Потоки и их взаимодействие

- **Поток покупателя** (`customer`): реализует логику поведения покупателя, включая генерацию списка покупок, переходы между отделами, ожидание в очереди и получение товаров
- **Поток продавца 1** (`seller1`): реализует логику обслуживания покупателей в отделе 1
- **Поток продавца 2** (`seller2`): реализует логику обслуживания покупателей в отделе 2
- **Главный поток** (`main`): создает все потоки, управляет жизненным циклом магазина, обрабатывает сигналы завершения

Взаимодействие потоков:

- Покупатели и продавцы взаимодействуют через разделяемую структуру данных `shop_t`, содержащую очереди для каждого отдела

- Синхронизация доступа к общим данным осуществляется через мьютексы
- Ожидание и пробуждение потоков реализовано через семафоры (в *Program_4-8*) или условные переменные (в *Program_9-10*)
- Каждый покупатель имеет свой семафор/условную переменную для ожидания завершения обслуживания

Модель параллельных вычислений

В данной программе используется модель **"Производители и потребители"** (Producer-Consumer) в сочетании с моделью **"Клиенты и серверы"**.

- **Покупатели** выступают в роли клиентов, которые запрашивают услугу у продавцов
- **Продавцы** выступают в роли серверов, которые ожидают запросов от клиентов и обрабатывают их
- Очереди покупателей реализуют буфер между производителями (покупателями, создающими запросы) и потребителями (продавцами, обрабатывающими запросы)

Дополнительно используется модель **"Управляющий и рабочие"**:

- Главный поток (`main`) выступает в роли управляющего, который координирует работу всех потоков, распределяет ресурсы и собирает результаты

Входные данные программы

Программа поддерживает несколько способов запуска:

1. Ввод числа покупателей через командную строку:

```
./main N
```

где `N` - количество покупателей (от 1 до 100)

2. Ввод с указанием выходного файла:

```
./main -n N -o output.txt
```

где `N` - количество покупателей, `output.txt` - имя файла для записи логов

3. Ввод из конфигурационного файла:

```
./main -c config.txt -o output.txt
```

где `config.txt` - конфигурационный файл в формате `ключ=значение`

Формат конфигурационного файла

Конфигурационный файл имеет простой формат `ключ=значение` . Ключи:

- `num_customers` - количество покупателей (*обязательный параметр*)
- `output` - имя файла для вывода (*опциональный параметр*)

Пример конфигурационного файла:

```
num_customers=5
output=result.txt
```

Диапазоны параметров

- **Количество покупателей:** от 1 до 100 (определяется константой `MAX_CUSTOMERS`)
- **Длина списка покупок:** от 1 до 5 товаров (генерируется случайно для каждого покупателя)
- **Выбор отдела:** 1 или 2 (генерируется случайно для каждого товара в списке)
- **Время обслуживания:** от 1 до 2 секунд (генерируется случайно для каждого обслуживания)

Структура проекта

IDZ_4/	
— README.md	# Данный файл
— Program_4-8/	# Программа на 4-8 баллов
— main.c	# Главный файл программы
— shop.h	# Заголовочный файл с определениями
— customer.c	# Реализация потока покупателя
— seller1.c	# Реализация потока продавца отдела 1
— seller2.c	# Реализация потока продавца отдела 2
— config1.txt	# Пример конфигурационного файла

	└─ [результаты тестирования]	# Файлы с результатами работы программы
	└─ Program_9-10/	# Программа на 9-10 баллов
	└─ main.c	# Главный файл программы
	└─ shop.h	# Заголовочный файл с определениями
	└─ customer.c	# Реализация потока покупателя
	└─ seller1.c	# Реализация потока продавца отдела 1
	└─ seller2.c	# Реализация потока продавца отдела 2
	└─ config1.txt	# Пример конфигурационного файла
	└─ [результаты тестирования]	# Файлы с результатами работы программы

Компиляция и запуск

Компиляция Program_4-8

```
cd Program_4-8
gcc main.c seller1.c seller2.c customer.c -o main -pthread
```

Компиляция Program_9-10

```
cd Program_9-10
gcc main.c seller1.c seller2.c customer.c -o main -pthread
```

Примеры запуска

Program_4-8:

```
# Запуск с 5 покупателями, вывод только в консоль
./main 5

# Запуск с выводом в файл
./main -n 5 -o output.txt

# Запуск с конфигурационным файлом
./main -c config1.txt -o output.txt
```

Program_9-10:

```
# Аналогичные команды
./main 5
./main -n 5 -o output.txt
./main -c config1.txt -o output.txt
```

Описание реализации

1. Program_4-8: реализация на семафорах и мьютексах

Используемые синхропримитивы:

- `pthread_mutex_t` - мьютекс для защиты разделяемых данных (очереди, флаги)
- `sem_t` - семафоры для синхронизации потоков:
 - `seller1_sem`, `seller2_sem` - семафоры продавцов (пробуждают продавцов при появлении покупателей)
 - `customer_sems[]` - массив семафоров для каждого покупателя (пробуждают покупателей после обслуживания)

Основные идеи:

1. Покупатель добавляет себя в очередь под защитой мьютекса и сигнализирует продавцу через `sem_post()`
2. Продавец ожидает сигнала через `sem_wait()` и обрабатывает покупателей из очереди
3. После обслуживания продавец сигнализирует конкретному покупателю через его семафор
4. Все операции с разделяемыми данными защищены мьютексом

2. Program_9-10: реализация на условных переменных, мьютексах и атомарных переменных

Используемые синхропримитивы:

- `pthread_mutex_t` - мьютексы для защиты разделяемых данных
- `pthread_cond_t` - условные переменные для синхронизации:
 - `seller1_cond`, `seller2_cond` - условные переменные продавцов
 - `customer_conds[]` - массив условных переменных для каждого покупателя
- `atomic_int` - атомарная переменная для флага аварийного завершения (`shutdown_flag`)

Основные идеи:

1. Покупатель добавляет себя в очередь под защитой мьютекса и сигнализирует продавцу через `pthread_cond_signal()`
2. Продавец ожидает сигнала через `pthread_cond_timedwait()` (с таймаутом для периодической проверки условий завершения)
3. После обслуживания продавец устанавливает флаг и сигнализирует покупателю через его условную переменную
4. Атомарная переменная `shutdown_flag` используется для безопасной установки флага завершения из обработчика сигналов

Отличия от Program_4-8:

- Использование условных переменных вместо семафоров позволяет более гибко управлять ожиданием с проверкой условий
- Использование атомарных переменных для флага завершения обеспечивает потокобезопасность без дополнительных мьютексов
- Использование `pthread_cond_timedwait()` позволяет периодически проверять условия завершения без блокировки на неопределенное время

Сравнительный анализ программ Program_4-8 и Program_9-10

Идентичность поведения

Обе программы реализуют одинаковую логику работы магазина:

- Одинаковая структура данных для магазина (очереди, флаги)
- Одинаковая логика поведения покупателей и продавцов
- Одинаковые генераторы случайных данных (хотя используются разные функции)
- Одинаковая обработка сигналов завершения

Преимущества каждого подхода

Program_4-8 (семафоры):

- Более простой и понятный код
- Меньше накладных расходов на синхронизацию
- Прямолинейная логика ожидания/пробуждения

Program_9-10 (условные переменные):

- Более гибкое управление ожиданием с проверкой условий
- Возможность использования таймаутов для периодической проверки условий
- Атомарные переменные обеспечивают потокобезопасность без блокировок для простых операций

Результаты тестирования

При одинаковых входных данных (количество покупателей, seed для генератора) обе программы ведут себя одинаково:

- Одинаковое количество покупателей
- Одинаковые списки покупок (при одинаковом seed)
- Одинаковая последовательность событий
- Одинаковое время работы (с учетом случайных задержек обслуживания)

Результаты работы программы

Пример вывода программы

Открытие магазина...

Число покупателей: 2

Продавец 1 начал работу

Продавец 2 начал работу

~~~~~

Покупатель 1: Мой список покупок: 2

~~~~~

Покупатель 1: Встал в очередь 2 для товара 2

Продавец 2: Обслуживает покупателя 1

~~~~~

Покупатель 2: Мой список покупок: 1

~~~~~

Покупатель 2: Встал в очередь 1 для товара 1

Продавец 1: Обслуживает покупателя 2

Продавец 1: Закончил обслуживание покупателя 2

Покупатель 2: Купил товар из отдела 1

Покупатель 2: Завершил покупки

Продавец 2: Закончил обслуживание покупателя 1
Покупатель 1: Купил товар из отдела 2

Покупатель 1: Завершил покупки

Закрытие магазина...

Продавец 1 завершил работу

Продавец 2 завершил работу

Магазин закрыт!

Тестовые случаи

Тест 1: Малое количество покупателей

Входные данные: num_customers=2

Конфигурационный файл: config1.txt

Результат: Оба покупателя успешно завершают покупки, продавцы корректно обслуживают всех покупателей.

Тест 2: Среднее количество покупателей

Входные данные: num_customers=5

Результат: Все покупатели успешно совершают покупки, очереди работают корректно, нет взаимных блокировок.

Тест 3: Большое количество покупателей

Входные данные: num_customers=10

Результат: Программа корректно обрабатывает большое количество параллельных потоков, все покупатели завершают покупки.

Тест 4: Прерывание программы

Действие: Запуск программы и прерывание через Ctrl+C

Результат: Программа корректно обрабатывает сигнал, все потоки завершаются без ошибок, ресурсы освобождаются.

Соответствие требованиям

Общие требования ✓

- ✓ ФИО, группа, номер варианта, условие задачи (в начале отчета)
- ✓ Сценарий решаемой задачи
- ✓ Корректное завершение программы и обработка сигналов
- ✓ Параметры задаются через командную строку
- ✓ Результаты работы программы отражены в отчете

Требования на 4-5 баллов ✓

- ✓ Сценарий в терминах предметной области
- ✓ Описание модели параллельных вычислений
- ✓ Описание входных данных с вариативными диапазонами
- ✓ Реализация на семафорах и мьютексах (Program_4-8)
- ✓ Ввод данных из командной строки
- ✓ Описание генераторов случайных чисел
- ✓ Информативный вывод
- ✓ Комментарии в коде

Требования на 6-7 баллов ✓

- ✓ Генерация случайных данных
- ✓ Ввод из командной строки

Требования на 8 баллов ✓

- ✓ Вывод в файл и консоль
- ✓ Ввод из конфигурационного файла
- ✓ Управление через ключи командной строки
- ✓ Примеры входных и выходных файлов

Требования на 9-10 баллов ✓

- ✓ Альтернативное решение на других синхропримитивах (Program_9-10)
- ✓ Сравнительный анализ поведения программ
- ✓ Проверка идентичности поведения